JC2001: Introduction to Software Engineering

Lecture 10: Software Testing

Lecture Schedule: Week 8

Dr Zhenhua Yu zhenhua.yu@abdn.ac.uk

Dr Riazul Islam riazul.islam@abdn.ac.uk

Dr Shahzad Mumtaz shahzad.mumtaz@abdn.ac.ukc

Learning Objectives

- Understand the stages of testing from testing during development to acceptance testing by customers.
- Discuss techniques that help you choose test cases that are geared towards discovering program defects.
- Understand test-first development.
- Know about three distinct types of testing (component testing, system testing, and release testing);
- Compare between development testing and user testing.

Reference: Ian Sommerville, Software Engineering – Chapter 8 Software Testing, 10th edition, Pearson, 2015.

Lecture Outline

- Describe Development Testing.
- Introduce Test-first development.
- Describe Release Testing and User Testing.

Testing

- The purpose of testing is to show that a program does what it is intended to do and to discover program defects before it is put into use.

- When you test software, you execute a program using artificial data. Then, you check the results of a test run for errors, anomalies, or information about the program's non-functional attributes.

Objectives of Software Testing

The objectives of software testing are:

- Validation Testing: To demonstrate to the developer and the customer that the software **meets its requirements**. For custom or bespoke software, this means that there should be **at least one test for every requirement** in the requirements document. For generic software products, it means that there should be **tests for all of the system features** that will be included in the product release. You may also **test combinations of features** to check for unwanted interactions between them.

- Defect Testing: **Find inputs or input sequences** where the **behaviour** of the software is **incorrect, undesirable, or does not conform to its specification**. These are caused by defects (bugs) in the software. Defects include system crashes, unwanted interactions with other systems, incorrect computations, and data corruption.

Objectives of Software Testing

System Crashes: The program suddenly stops working, freezes, or closes unexpectedly — often showing an error or restarting.

Unwanted Interactions with Other Systems: The software interferes with or causes problems in another system when both are running together.

Incorrect Computations: The software gives wrong results or calculations — the math or logic is incorrect.

Data Corruption: Stored data becomes damaged, lost, or incorrect, usually because the software saves it wrongly.

Validation Testing

The goal of validation testing is to verify whether a software product meets its acceptance criteria. That is, you expect the system to perform correctly using a set of test cases that reflect the system's expected use.

Gmail login test case

Test Case ID	Pre-requisite	Test Case Description	Steps to reproduce
TC_001	The gmail link should be available	To verify if the login page opens successfully	Open the web browser and launch the following URL: https://www.gmail.com

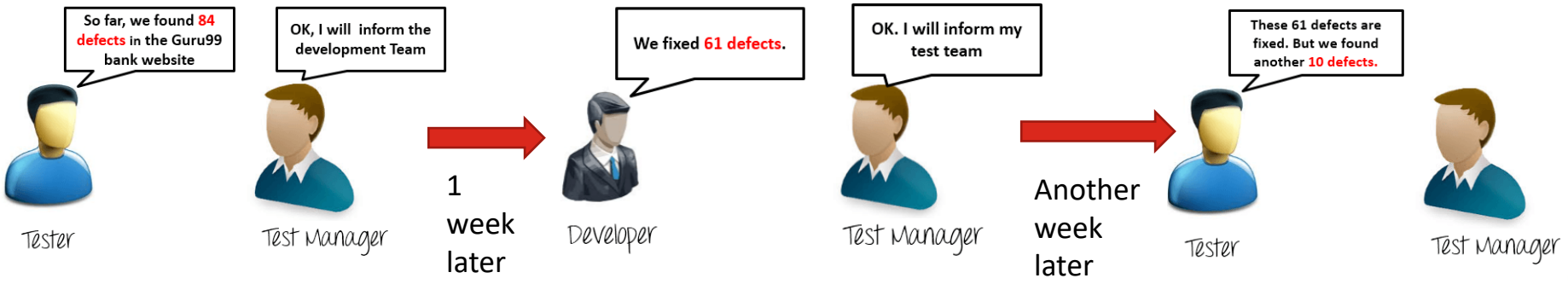
Expected Result	Actual Result	Pass/fail
The gmail webpage should display.	The gmail webpage was displayed.	Pass

Test Case ID	Pre-requisite	Test Case Description	Steps to reproduce
TC_002		To verify the error message when the Enter your email field is left blank	1.Launch the URL 2.Leave the fields empty and click on "Next" button

Expected Result	Actual Result	Pass/fail
The following error message should be displayed 'Enter an email or phone number'.	The following error message was displayed 'Enter an email or phone number'.	Pass

Defect Testing

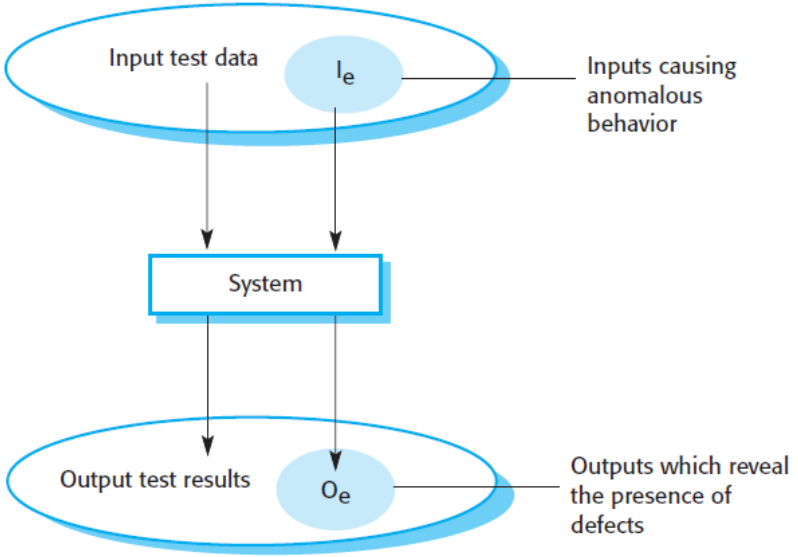
Defect testing is testing where test cases are designed to *expose defects*. The test cases in defect testing can be deliberately obscure and need not reflect how the system is normally used.



Reference: https://www.guru99.com/defect-management-process.html

Validation Testing vs. Defect Testing

- The system accepts inputs from some input set *I* and generates outputs in an output set *O*.
- Some of the outputs will be erroneous, these outputs are denoted O_e and are generated by the system in response to inputs in the set I_e.
- The priority in defect testing is to find those inputs in the set I_e because these reveal problems in the system.



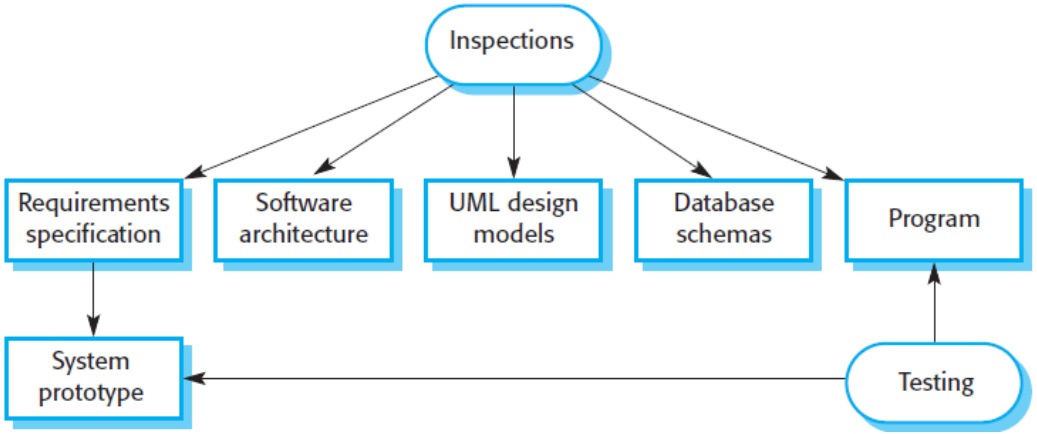
Validation testing involves testing with correct inputs that are outside I_e. These stimulate the system to generate the expected correct outputs.

Validation Testing vs. Defect Testing

System	Type of Test	Purpose	Example Outcome
ATM	Validation	Check login with valid card/PIN	Works correctly
ATM	Defect	Wrong PIN causes freeze	Defect found
E-commerce	Validation	Add item updates cart correctly	Works correctly
E-commerce	Defect	Large quantity crashes system	Defect found

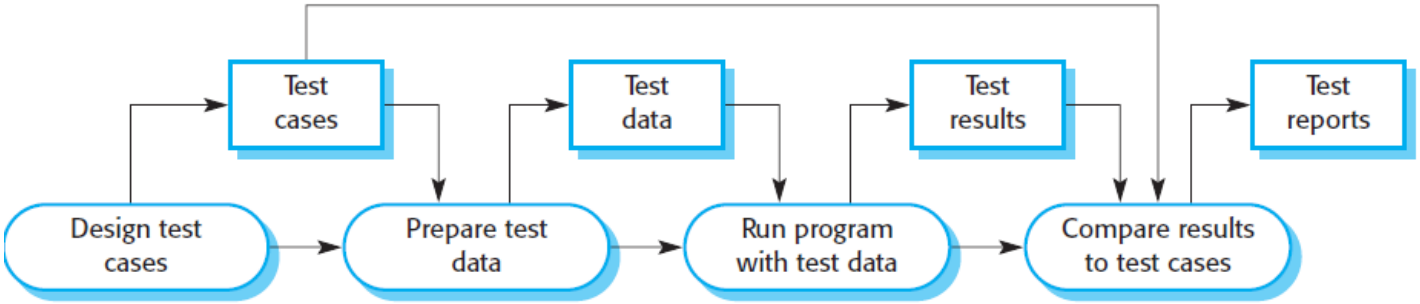
Inspections and Testing

Software inspection mostly focus on the source code of a system, but any readable representation of the software, such as its requirements or a design model, can be inspected. **When you inspect a system, you use knowledge of the system**, its application domain and the programming or modeling language to discover errors.



Software Testing Process Model

- **Test cases** are specifications of the **inputs** to the test and the **expected output** from the system, plus a **statement** of what is being tested.
- Test data are the inputs that have been devised to test a system. Test data can sometimes be generated automatically, but automated test case generation is impossible (combinatorial explosion of number of required test cases to produce a correct output).



Stages of Testing a Software System

A software system typically goes through three stages of testing:

- **Development Testing**, where the system is tested during development to discover bugs and defects. System designers and programmers are likely to be involved in the testing process.

- **Release Testing**, where a separate testing team tests a complete version of the system before it is released to users. The aim of release testing is to check that the system meets the requirements of the system stakeholders.

- **User Testing**, where users or potential users of a system test the system in their own environment. Acceptance testing is one type of user testing where the customer formally tests a system to decide if it should be accepted from the system supplier or if further development is needed.

Development testing

1495 UNIVERSITY OF ABERDEEN

Development Testing

There are three stages of development testing:

- **Unit testing**, where individual program units or object classes are tested. Unit testing should focus on testing the functionality of objects or methods.
- **Component testing**, where several individual program units are integrated to create composite components. Component testing should focus on *testing the component interfaces* that provide access to the component functions.
- **System testing**, where some or all of the components in a system are integrated and the system is tested as a whole. System testing should focus on *testing component interactions*.

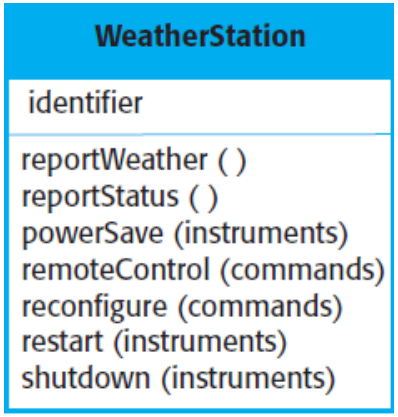
- Development testing is primarily a defect testing process, where the aim of testing is to discover bugs in the software.

Unit Testing

- Unit testing is the process of testing program components (such as methods or object classes) in isolation.
- It is a defect testing process.
- Individual functions or methods are the simplest type of component. A Unit test should include calls to these routines with different input parameters.
- When testing object classes, your Unit tests should be designed such that it covers all of the features of the object, i.e., simulate all events that cause a state change.

Weather Station Object Interface

- The WeatherStation object has a single attribute, which is its identifier. This is a constant that is set up when the weather station is installed. Therefore, you only need a test that checks if it has been properly set up.
- You need to define test cases for all of the methods associated with the WeatherStation object. Each method should be tested in isolation, but in some cases, test sequences are necessary. For example, to test shutdown(), you need to have executed restart().



Examples of state sequences

Shutdown → Running → Shutdown
Configuring → Running → Testing → Transmitting → Running
Running → Collecting → Running → Summarizing → Transmitting → Running

Automated Unit Test

An automated unit test has three parts:

- **Setup Part**, where you initialize the system with the test case, namely, the inputs and expected outputs.
- **Call Part**, where you call the object or method to be tested.
- **Assertion Part**, where you compare the result of the call with the expected result. If the assertion evaluates to true, the test has been successful; if false, then it has failed.

Mock objects

- Sometimes, the object that you are testing has **dependencies** on other objects that may not have been implemented, or whose use slows down the testing process. For example, if an object calls a database, this may involve a slow setup process before it can be used. In such cases, you may decide to use mock objects.

- Mock objects are objects with the same interface as the external objects being used to simulate its functionality. For example, a mock object simulating a database may have only **a few data items that are organised in an array**. They can be assessed quickly, without the overheads of calling a database and accessing disks.

Mock objects

- Mock objects can also be used to simulate abnormal operations or rare events. For example, if your system is intended to take action at certain times of day, your mock object can simply return those times, irrespective of the actual clock time.

Choosing unit test cases

Testing is expensive and time consuming. Thus, it is important that you choose effective unit test cases. Effectiveness means:

- The test cases should show that, when used as expected, the component that you are testing does what it is supposed to do.
- If there are defects in the component, these should be revealed by test cases.

Two types of test cases should be designed.

- The first type should reflect normal operation of a program and should show how the component works.
- The second type should be based on testing experience of where common problems arise. It should use abnormal inputs to check that these are properly processed and do not crash the component.

Strategies for Choosing Test Cases

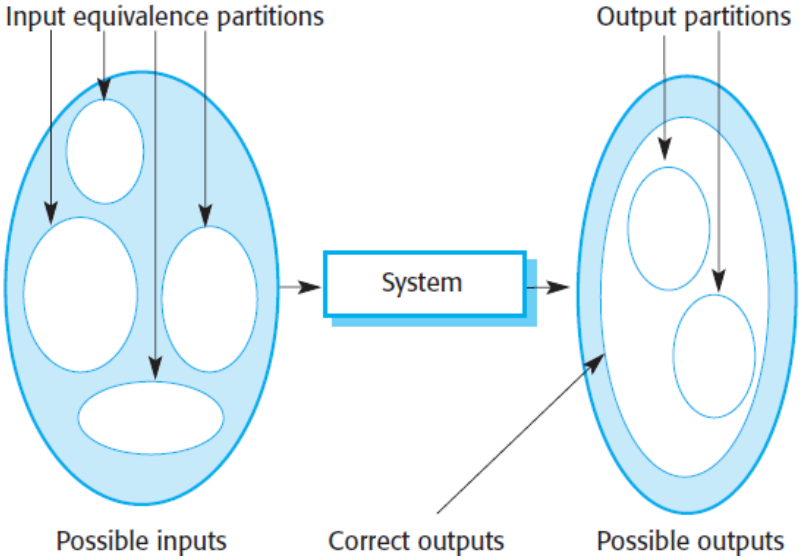
Two strategies that can be effective in helping you choose test cases are:

- **Partition testing**, where you identify groups of inputs that have common characteristics and should be processed in the same way. You should choose tests from within each of these groups.

- **Guideline-based testing**, where you use testing guidelines to choose test cases. These guidelines reflect previous experience of the kinds of errors that programmers often make when developing components.

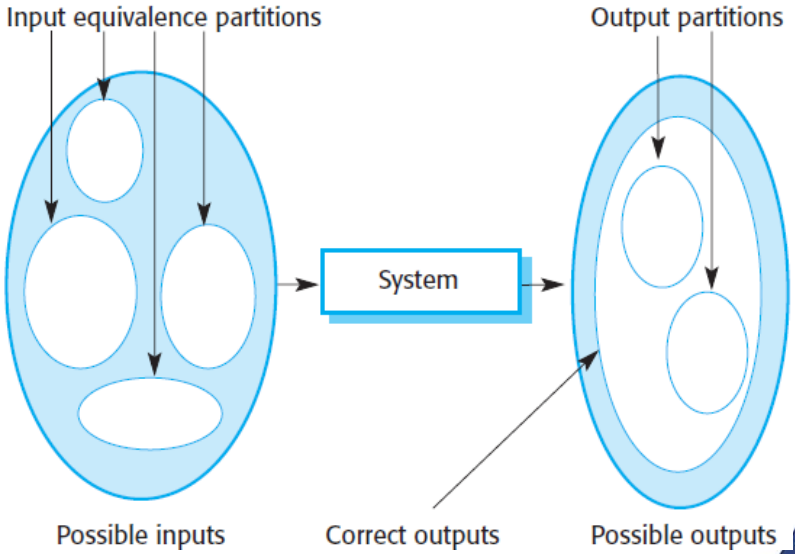
Partition Testing Strategy

- Input data and output results of a program can be thought of as members of sets with common characteristics. For example, positive numbers and negative numbers. If you test a program with two positive numbers, you would expect it to behave the same way with all positive numbers.
- A systematic approach to test-case design is based on identifying all input and output partitions for a system or component. Test cases are designed so that the input or outputs lie within these partitions.



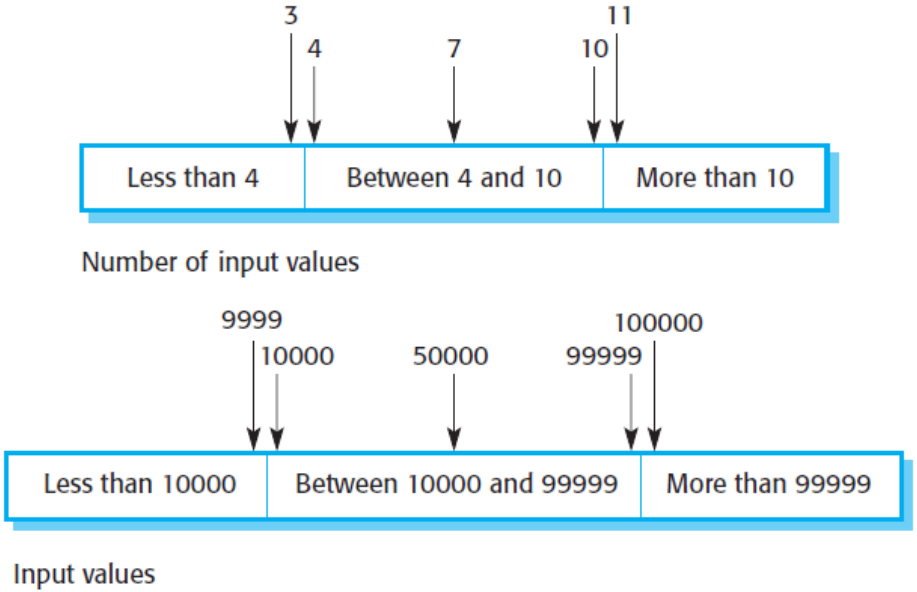
Partition Testing Strategy

- The large shaded ellipse on the left represents the set of all possible inputs to the program that is being tested. The smaller unshaded ellipses represent equivalence partitions.
  - The shaded area in the left ellipse represent inputs that are invalid.
  - The shaded area in the right ellipse represent exceptions that may occur, that is, responses to invalid inputs.
- Once a set of partitions are identified, you choose test cases from each of these partitions. A good rule of thumb is to choose test cases on the boundaries of these partitions.



Partition Testing Strategy

Boundary values are often atypical. For example, zero may behave differently from other non-negative numbers. These are often overlooked by programmers. Thus, program failures occur when processing these atypical values.



Guidelines-based testing

Equivalence partitioning is an effective approach to testing because it helps account for errors that programmers often make when processing inputs at the edges of partitions. You can also use testing guidelines to help choose test cases. For example, when you are testing programs with sequences, arrays, or lists, guidelines that could help reveal defects include:

- Test software with sequences that have only one single value. Programmers tend to think of sequences as made up of several values. So if presented with a single value sequence, a program may not work correctly.
- Use different sequences of different sizes in different tests. This decreases the chances that a program with defects will accidentally produce a correct output because of some accidental characteristics of the input.
- Derive tests so that the first, middle, and last elements of the sequence are accessed. This approach reveals problems at partition boundaries.

Guidelines-based testing

Some general guidelines for test-case design include:

- Choose inputs that force the system to generate all error messages:
- Design inputs that cause input buffers to overflow.
- Repeat the same input or series of inputs numerous times.
- Force invalid outputs to be generated.
- Force computation results to be too large or too small.

Component Testing

- Software components are often made up of several interacting objects.
- You access the functionality of these objects through component interfaces.
- Testing composite components should therefore focus on showing that the component interface or interfaces behave according to specification.
- At this point of the testing process, you can assume that unit tests on the individual objects within the component have been completed.

Interface Testing

There are different types of interfaces between program components and, consequently, different types of interface error that can occur:

- **Parameter interfaces**. These are interfaces in which data or sometimes function references are passed from one component to another. Methods in an object have a parameter interface.

- **Shared memory interfaces**. These are interfaces in which a block of memory is shared between components. Data is placed in memory by one component and retrieved from there by other components. Commonly used in embedded systems, e.g., sensors that generate data that is retrieved and processed by other system components.

Interface Testing

There are different types of interfaces between program components and, consequently, different types of interface error that can occur (cont'd):

- **Procedural interfaces**. These are interfaces in which one component encapsulates a set of procedures that can be called by other components.

- **Message passing interfaces**. These are interfaces in which one component requests a service from another component by passing a message to it. A return message includes the results of executing the service. Commonly found in client-server systems, service-oriented architectures.

Interface Errors

Interface errors fall into three groups:

- **Interface misuse**. A calling component calls some other component and makes an error in the use of its interface. This type of error is common in parameter interfaces, where parameters may be of the wrong type or be passed in the wrong order, or the wrong number of parameters may be passed.
- **Interface misunderstanding**. A calling component misunderstands the specification of the interface of the called component and makes assumptions about its behaviour. For example, a binary search method may be called with a parameter that is an unordered array. The search would then fail.
- **Timing errors**. These occur in real-time systems that use a shared memory or a message passing interface. The producer of data and consumer of data operate at different speeds. The consumer can access out-of-date information because the producer of the information has not updated the shared interface specification.

Interface Testing: General Guidelines

Some general guidelines for interface testing are:

- Design tests such that parameters to a called procedure are at the extreme ends of their ranges.
- Always test pointer parameters with null pointers.
- Design tests which cause the component to fail.
- Use stress testing in message passing systems.
- In shared memory systems, vary the order in which components are activated.

System Testing

System testing involves integrating components to create **a version of the system** and then testing the integrated system. System testing checks that components are compatible, interact correctly, and transfer the right data at the right time across their interfaces. It **overlaps with component testing**, but there are two differences:

- During system testing, **reusable components** that have been separately developed and off-the-shelf systems may be integrated with newly developed components. The complete system is then tested.
- Components **developed by different team members** or subteams may be integrated at this stage. System testing is a collective rather than an individual process.

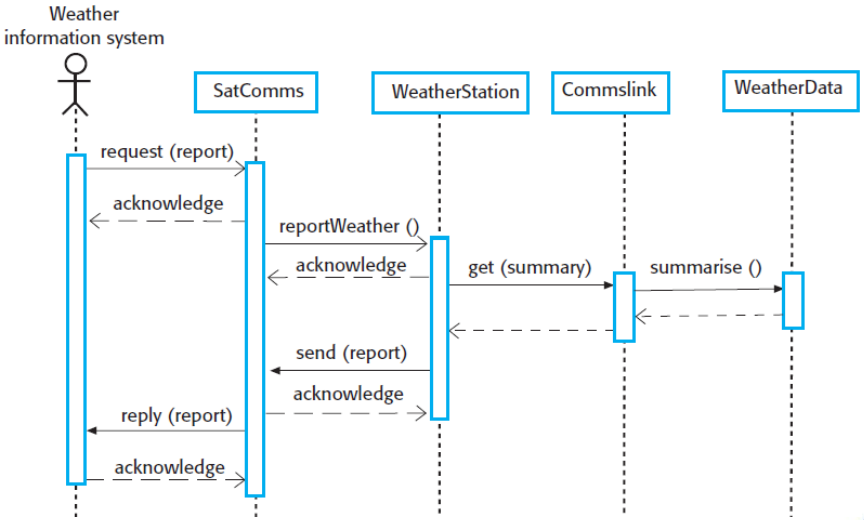
System Testing

- System testing focus on testing the interactions between the components and objects that make up the system. This interaction testing should discover component bugs that are only revealed when **a component is used by other components** in the system. Because of its focus on interactions, ***use-case based testing*** *is an effective approach to system testing*. Testing the use cases forces these interactions to occur.

Wilderness Station Example

- You can use this sequence diagram **to identify operations** that will be tested and to help design the test cases to execute the tests.

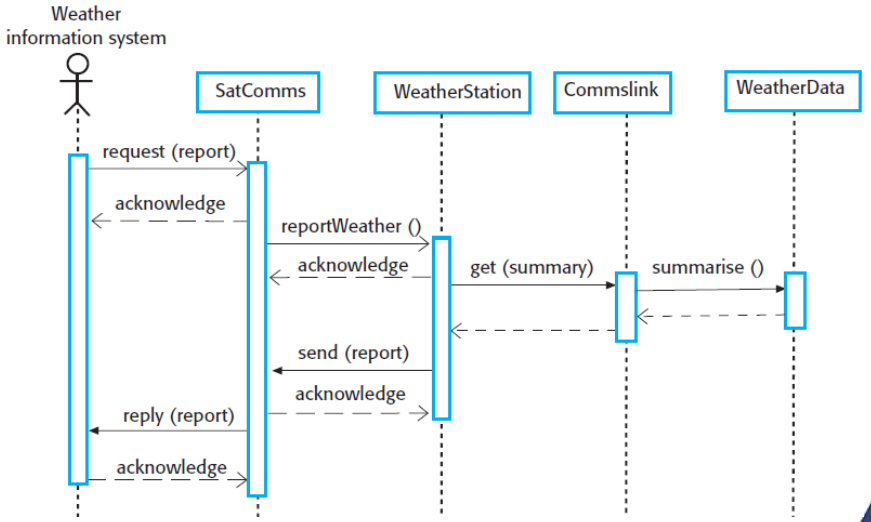
- For example, issuing a request for a report will result in the execution of the following thread of methods:



SatComms:request → WeatherStation:reportWeather → Commslink:Get(summary) → WeatherData:summarize

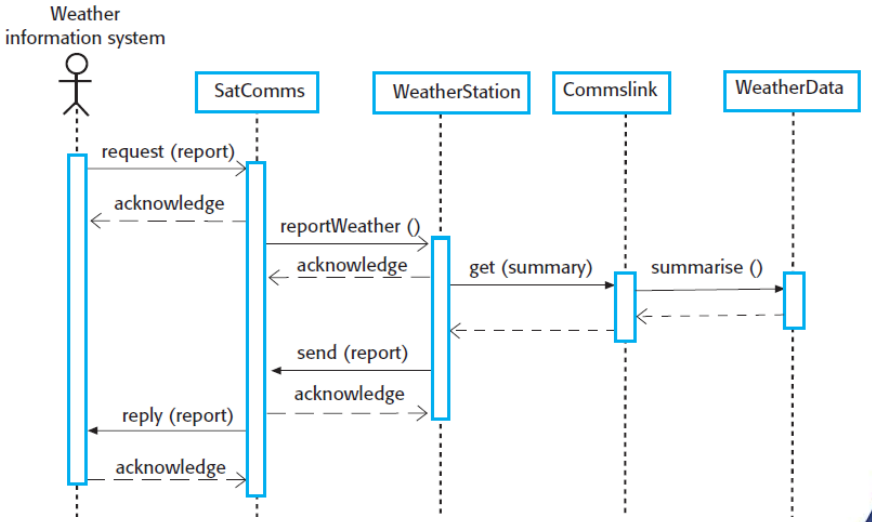
Wilderness Station Example

- The sequence diagram helps you to design the specific test cases, as it shows what inputs are required and what outputs are created:
  - An input of a request for a report should have an associated acknowledgement. A report should ultimately be returned from the request. During testing, you should create summarised data that can be used to check that the report is correctly organised.



Wilderness Station Example

- The sequence diagram helps you to design the specific test cases, as it shows what inputs are required and what outputs are created (cont'd):
    - An input request for a report to WeatherStation results in a summarised report being generated. You can test this in isolation by creating raw data corresponding to the summary that you have prepared for the test of SatComms and checking that the WeatherStation object correctly produces this summary.



Guidelines for choosing test cases

Exhaustive testing, where every possible program execution sequence is tested, is impossible. Thus, testing has to be based on a subset of possible test cases. Some software companies have policies for choosing this subset. Some guidelines for choosing a subset of test cases include:

- All system functions that are accessed through menus should be tested.
- Combinations of functions that are accessed through the same menu must be tested.
- Where user input is provided, all functions must be tested with both correct and incorrect input.

Test-driven development



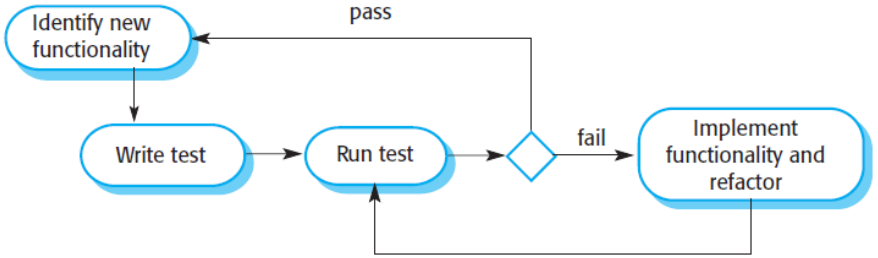
1495 UNIVERSITY OF ABERDEEN

Test-driven development

- Test-driven development (TDD) is an approach to program development in which you interleave testing and code development.
- Tests are written before code and ‘passing’ the tests is the critical driver of development.
- You develop code incrementally, along with a set of tests for that increment. You don’t start working on the next increment until the code that you have developed passes all of its tests.

- TDD was introduced as part of the XP agile development method. However, it has now gained mainstream acceptance and may be used in both agile and plan-based approaches.

TDD process



- The steps in the process are as follows:
  - Start by identifying the increment or functionality that is required. This should normally be small and implementable in a few lines of code.
  - Write a test for this functionality and implement it as an automated test. This means that the test can be executed and will report whether or not it has passed or failed.
  - Run the test, along with all other tests that have been implemented. Initially, you have not implemented the functionality, so the new test will fail. This is deliberate as it shows that the test adds something to the test set.
  - Implement the functionality and re-run the test. This may involve refactoring existing code to improve it and add new code to what's already there.
  - Once all tests run successfully, move on to implement the next chunk of functionality.

Benefits of test-driven development

- The benefits of TDD are:
  - **Code coverage**. In principle, every code segment that you write should have at least one associated test. Thus, you can be confident that all of the code in the system has actually been executed. Code is tested as it is written, so defects are discovered early in the development process.
  - **Regression testing**. A test suite is developed incrementally as a program is developing. You can always run regression tests to check that changes to the program have not introduced new bugs.
  - **Simplified debugging**. When a test fails, it should be obvious where the problem lies. The newly written code needs to be checked and modified. You do not need to use debugging tools to locate the problem.
  - **System documentation**. The tests themselves act as a form of documentation that describe what the code should be doing. Reading the tests can make it easier to understand the code.

Release testing

1495 UNIVERSITY OF ABERDEEN

Release Testing

Release testing is the process of testing **a particular release of a system** that is intended for use outside of the development team. **Normally**, the system release is for **customers** and users. In a **complex project**, the release could be for **other teams** that are developing related systems. There are two important distinctions between *release testing* and *system testing*:

- The system development team should not be responsible for release testing
- **Release testing** is a process of validation checking to ensure that a system meets its requirements and is good enough for use by system customers. **System testing** by the development team should focus on discovering bugs in the system.

Release Testing Process

The goal of the release testing process is to convince the supplier of the system that it is good enough for use. Thus, release testing has to show that the system delivers its specified functionality, performance, and dependability, and that it does not fail during normal use. Release testing can be categorised into three different approaches:

- Requirements-based testing.
- Scenario testing.
- Performance testing.

Requirements-based Testing

- A general principle of good requirements engineering practice is that **requirements should be testable**.

- Requirements-based testing is a systematic approach to test-case design where you consider **each requirement and derive a set of tests for it**.

- Requirements-based testing is **validation rather than defect testing** – you are trying to demonstrate that the system has properly implemented its requirements.

Example: Mentcare system

If a patient is known to be allergic to any particular medication, then prescription of that medication shall result in a warning message being issued to the system user.

If a prescriber chooses to ignore an allergy warning, he or she shall provide a reason why this has been ignored.

To check if these **requirements** have been satisfied, several related tests must be developed:

- Setup a patient record with no known allergies. Prescribe medication for allergies that are known to exists. Check that a warning message is not issued by the system.
- Setup a patient record with a known allergy. Prescribe medication that the patient is allergic to and check that the warning is issued by the system.
- Setup a patient record in which allergies to two or more drugs are recorded. Prescribe both of these drugs separately and check that the correct warning for each drug is issued.

Example: Mentcare system

If a patient is known to be allergic to any particular medication, then prescription of that medication shall result in a warning message being issued to the system user.

If a prescriber chooses to ignore an allergy warning, he or she shall provide a reason why this has been ignored.

To check if these requirements have been satisfied, several related tests must be developed (cont'd):

- Prescribe two drugs that the patient is allergic to. Check that two warnings are correctly issued.

- Prescribe a drug that issues a warning and overrule that warning. Check that the system requires the user to provide information explaining why the warning was overruled.

Scenario Testing

- Scenario testing is **an approach to release testing** whereby you devise typical scenarios of use and use these scenarios to develop test cases for the system.
- A scenario is **a story** that describes one way in which the system might be used.
- Scenarios **should be realistic**, and real system users should be able to relate to them. If you have used scenarios or **user stories** as part of the requirements engineering process, then you may be able to reuse them as testing scenarios.

Scenario Testing

- When you use a scenario-based approach, you are normally testing **several requirements within the same scenario**. Thus, as well as checking individual requirements, you are also checking that combinations of requirements do not cause problems.

Example: Mentcare system

George is a nurse who specializes in mental health care. One of his responsibilities is to visit patients at home to check that their treatment is effective and that they are not suffering from medication side effects.

On a day for home visits, George logs into the Mentcare system and uses it to print his schedule of home visits for that day, along with summary information about the patients to be visited. He requests that the records for these patients be downloaded to his laptop. He is prompted for his key phrase to encrypt the records on the laptop.

One of the patients whom he visits is Jim, who is being treated with medication for depression. Jim feels that the medication is helping him but believes that it has the side effect of keeping him awake at night. George looks up Jim's record and is prompted for his key phrase to decrypt the record. He checks the drug prescribed and queries its side effects. Sleeplessness is a known side effect, so he notes the problem in Jim's record and suggests that he visit the clinic to have his medication changed. Jim agrees, so George enters a prompt to call him when he gets back to the clinic to make an appointment with a physician. George ends the consultation, and the system re-encrypts Jim's record.

After finishing his consultations, George returns to the clinic and uploads the records of patients visited to the database. The system generates a call list for George of those patients whom he has to contact for follow-up information and make clinic appointments.

A scenario example

Example: Mentcare system

This scenario describes one way that the system may be used on a home visit. It tests a number of features of the Mentcare system:

- Authentication by logging on to the system.
- Downloading and uploading of specified patient records to a laptop.
- Home visit scheduling.
- Encryption and decryption of patient records on a mobile device.
- Records retrieval and modification.
- Links with the drugs database that maintains side-effect information.
- The system for call prompting.

Performance Testing

- Once a system has been completely integrated, it is possible to test for emergent properties, such as performance and reliability. Performance tests have to be designed to ensure that the system can process its intended load.
- This usually involve running a series of tests where you increase the load until the system performance becomes unacceptable. This approach is also known as **stress testing**. Stress testing helps to:
  - Test the failure behaviour of the system. Stress testing checks that overloading the system causes it to ``fail-soft'' rather than collapse under its load.
  - Reveal defects that only show up when the system is fully loaded. Although it can be argued that these defects are unlikely to cause system failures in normal use, there may be unusual combinations of circumstances that the stress testing replicates.

User Testing



1495 UNIVERSITY OF ABERDEEN

User Testing

User or customer testing is a stage in the testing process in which users or customers provide input and advice on system testing. There are three types of user testing:

- **Alpha testing**, where a selected group of software users work closely with the development team to test early releases of the software.
- **Beta testing**, where a release of the software is made available to a larger group of users to allow them to experiment and to raise problems that they discover with the system developers.
- **Acceptance testing**, where customers test a system to decide whether or not it is ready to be accepted from the system developers and deployed in the customer environment.

Alpha testing

- In alpha testing, users and developers work together to test a system as it is being developed. This means that the users can identify problems and issues that are not readily apparent to the development testing team.

- Alpha testing is often used when developing software products, e.g., computer games, or apps. Experienced users of these products may be willing to get involved in the alpha testing process because it gives them early information about new system features that they can exploit. It also reduces the risk that unanticipated changes to the software will have disruptive effects on their business.

Beta Testing

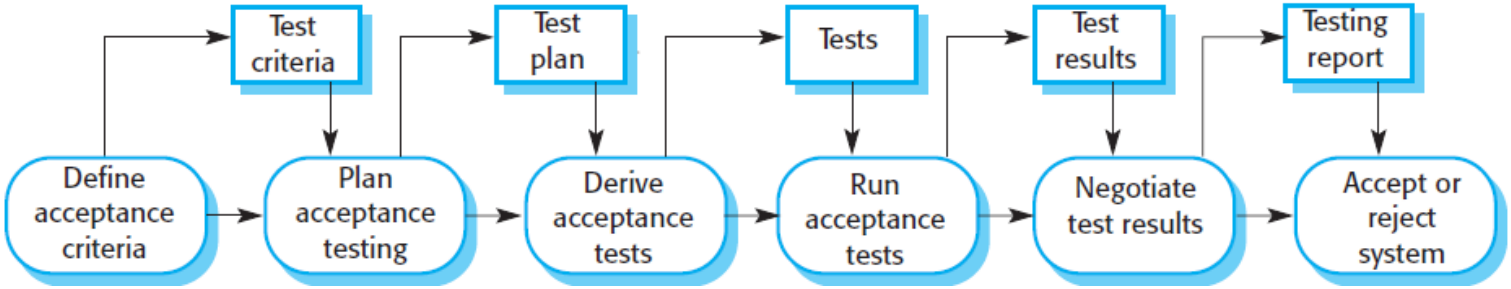
- Beta testing takes place when an early, sometimes unfinished, release of a software system is made available to a larger group of customers and users for evaluation.

- Beta testers may be a selected group of customers who are early adopters of the system.

- Alternatively, the software may be made publicly available for use by anyone who is interested in experimenting with it.

Acceptance Testing

Acceptance testing is an inherent part of custom systems development. Customers test a system, using their own data, and decide if it should be accepted from the system developer. Acceptance implies that final payment should be made for the software.



Acceptance Testing Process

There are six stages in the acceptance testing process:

- **Define acceptance criteria**. This stage should ideally take place early in the process before the contract for the system is signed.
- **Plan acceptance testing**. This stage involves deciding on the resources, time, and budget for acceptance testing and establishing a testing schedule.
- **Derive acceptance tests**. Once the acceptance criteria have been established, tests have to be designed to check whether or not a system is acceptable. Acceptance tests should aim to test both the functional and non-functional requirements.
- **Run acceptance tests**. The agreed acceptance tests are executed on the system.

Acceptance Testing Process

There are six stages in the acceptance testing process (cont'd):

- **Negotiate test results**. It is unlikely that all of the defined acceptance tests will pass and that there will be no problems with the system. If this is the case, then acceptance testing is complete and handed over. However, some problems will be discovered, so the customer and developer have to negotiate to decide if the system is good enough to be used.
- **Reject / accept system**. This stage involves a meeting between the developers and the customer to decide on whether or not the system should be accepted. If the system is not good enough for use, then further development is required to fix the identified problems. Once complete, the acceptance testing phase is repeated.

User Testing Comparison

Type	Who tests	When	Where	Example
Alpha	Small group of users + developers	Early stage	Developer's site	Game tested by 10 players at company
Beta	Large group of real users	Later stage (almost ready)	Users' own environment	Public beta version of Windows
Acceptance	Customer/client	Final stage	Customer's workplace	Hospital tests final patient record system

Tools and Software Programs

Functional Testing Tools: Check if the software works as expected.

Performance and Stress Testing Tools: Test how the system performs under heavy load or stress.

Security Testing Tools: Check for vulnerabilities, data leaks, or weak authentication.

Continuous Testing / CI Tools: Run tests automatically whenever new code is added.

Testing Type	Purpose	Tools / Software
Unit Testing	Test small pieces of code (like individual functions or methods).	JUnit (Java), PyTest (Python), NUnit (.NET)
Integration Testing	Test if different modules or components work together correctly.	TestNG, Postman (for API integration), SoapUI
System Testing	Test the complete system as a whole.	Selenium, Cypress, Robot Framework
Acceptance Testing	Check if the system meets user/business requirements.	Cucumber, FitNesse
Regression Testing	Re-test after changes to make sure old features still work.	Selenium, Jenkins (automated testing)
Performance / Stress Testing	Check how system behaves under heavy load or pressure.	JMeter, LoadRunner, Locust
Security Testing	Find vulnerabilities and weaknesses.	OWASP ZAP, Burp Suite, Nessus
Usability Testing	See how easy and friendly the system is for users.	Crazy Egg, Hotjar, UserTesting
Mobile App Testing	Test Android/iOS applications.	Appium, Espresso
Continuous Integration (CI) Testing	Automatically run tests whenever new code is added.	Jenkins, GitHub Actions, GitLab CI/CD

Lessons Learned

- Testing can only show the presence of errors in a program. It cannot show that there are no remaining faults.
- Development testing is the responsibility of the software development team. A separate team should be responsible for testing a system before it is released to customers. In the user testing process, customers or system users provide test data and check that tests are successful.
- Development testing includes unit testing in which you test individual objects and methods, component testing in which you test related groups of objects, and system testing in which you test partial or complete systems.

Lessons Learned

- Test-driven development is an approach to development where tests are written before the code is to be tested. Small code changes are made, and the code is refactored until all tests execute successfully.
- Scenario testing is useful because it replicates the practical use of the system. It involves inventing a typical usage scenario and using this to derive test cases.
- Acceptance testing is a user testing process in which the aim is to decide if the software is good enough to be deployed and used in its planned operational environment.